

CS 527 Research Project Report: Group-2

Ajmal Ali, Muneeb Hoda
University of Illinois at Urbana-Champaign
[aali274,shoda6]@illinois.edu

Abstract

Generative Web interfaces are increasingly being used to build interactive applications, but ensuring their behavioral correctness remains a challenge. Existing testing techniques for Web interfaces primarily focus on structural validation (e.g., DOM correctness), visual regression, or isolated functional checks. However, such approaches provide limited support for reasoning about relational dependencies across multi-step interaction flows. As a result, subtle behavioral flaws—such as improper authentication enforcement, inconsistent validation handling, incorrect redirects, or incoherent state transitions may remain undetected.

In this proposal, we introduce a behavioral consistency testing approach that models generative Web UIs as state-transition systems and verifies relational invariants across interaction paths. Rather than validating individual UI actions in isolation, our technique analyzes logical relationships between states, including precondition enforcement, path equivalence, and invariant preservation under repeated or alternative interactions. By defining relational behavioral oracles that capture cross-state constraints, our approach aims to uncover inconsistencies that traditional structural and visual testing methods fail to detect.

1 Artifact Information

Link to the GitHub Repository: [put the link here]

2 Introduction

Web interfaces are foundational components of modern software, mediating interaction between users and services across domains such as e-commerce, enterprise dashboards, and data-centric applications. Ensuring the behavioral correctness of these interfaces extends beyond verifying isolated UI elements; it requires reasoning about how sequences of interactions unfold. Incorrect or inconsistent interaction logic can lead to broken workflows, security vulnerabilities, and degraded user experience, making rigorous behavioral analysis an essential aspect of software quality assurance.

Existing techniques for testing Web UIs include structural and functional testing frameworks that verify control presence and correctness, often via scripted interactions. Model-based testing (MBT) abstracts UI behavior into state models to systematically generate tests across interaction paths [4]. Invariant-based GUI testing automatically verifies properties of interface states against expected logical constraints [3]. Dynamic GUI testing techniques explore execution paths at runtime to uncover behavioral issues across event sequences [1]. More recent research has leveraged machine learning and vision models to enhance automated GUI testing coverage and effectiveness [5]. While these methods improve coverage and detect certain classes of faults, they typically examine interactions in isolation or focus on structural properties rather than relational behavioral properties across multiple paths.

Despite these advances, a significant gap remains in capturing **relational behavioral correctness**—the logical relationships among interaction paths in a UI. Current approaches do not inherently ensure that constraints such as “authentication must precede restricted access” or “invalid input should prevent forward navigation” hold across the space of possible interaction sequences. Model extraction methods generate abstract graphs of actions and states, but do not automatically verify relations across paths. As a result, logically inconsistent behaviors may remain undetected without manual specification of properties or exhaustive scripting.

To address this gap, we propose a **behavioral consistency testing framework** that models a generative Web UI as a state-transition system inferred from interaction traces and uses relational invariants across interaction paths to capture logical expectations between states. Rather than validating individual actions in isolation, our approach analyzes cross-path relationships such as preservation of preconditions, equivalence of alternative workflows, and prevention of forbidden transitions. By enforcing relational behavioral oracles over inferred paths, we aim to detect subtle logical flaws that structural and isolated functional tests cannot uncover.

Our experimental plan applies this framework to representative generative Web UIs with complex interactive logic, including authentication flows, conditional navigation, and validation logic. We will evaluate the framework’s ability to automatically identify logical inconsistencies compared to structural and model-based baselines. Metrics include detection rate of injected faults, comparison with baseline methods, and analysis of scalability as UI complexity increases.

Preliminary experiments suggest that relational behavioral testing reveals logical inconsistencies missed by structural automation or visual regression. For example, equivalent paths that conceptually represent the same workflow sometimes lead to divergent outcomes, and validation failures may allow undesired progression under certain sequences, underscoring the need for relational reasoning.

The primary contributions of this work are:

- A methodology for automatically inferring behavioral state models from interaction traces.
- A behavioral consistency testing framework that defines and enforces relational invariants across interaction paths.
- An empirical evaluation demonstrating the effectiveness of the proposed approach in identifying logical inconsistencies beyond traditional testing methods.
- A set of behavioral properties and benchmarks to facilitate further research into relational UI behavioral testing.

3 Proposed Technique

3.1 Approach Overview

Our framework systematically analyzes relational behavioral correctness in Web UIs through a multi-stage pipeline that combines automated exploration, state abstraction, model inference, and relational invariant checking.

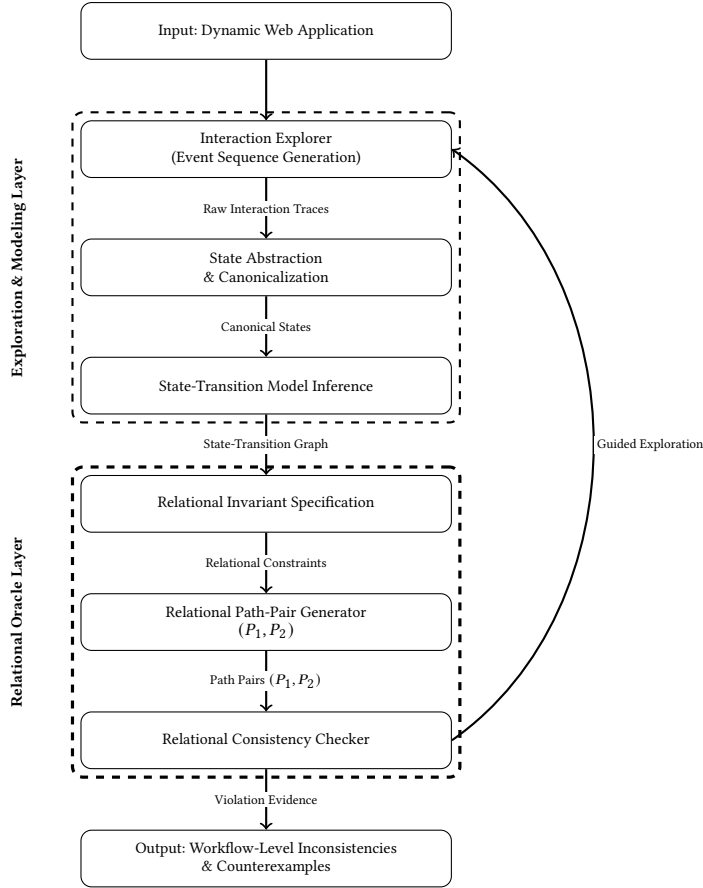


Figure 1: Relational behavioral consistency testing framework.

1. Interaction Trace Collection. We begin by automatically exploring the target Web UI using a dynamic exploration engine. The explorer generates sequences of user events (clicks, form submissions, navigation actions) and records execution traces consisting of: (i) event sequences, (ii) DOM snapshots, (iii) URL and session metadata, (iv) observable state changes.

Unlike purely coverage-driven approaches, our objective is not only to discover new states but to collect sufficiently diverse interaction paths for relational reasoning.

2. State Abstraction and Canonicalization. Raw DOM snapshots often contain noise such as timestamps, dynamic identifiers, and transient content. We apply abstraction techniques to construct canonical UI states by filtering non-semantic variations. States are

represented using structural fingerprints combined with contextual metadata (URL, authentication status, visible widgets).

This abstraction enables grouping semantically equivalent states reached through different paths.

3. State-Transition Model Inference. From the abstracted traces, we infer a state-transition graph where: nodes represent canonical UI states, edges represent user-triggered transitions labeled with events and parameters.

The resulting model captures the reachable behavioral structure of the UI.

4. Relational Invariant Specification. We define relational behavioral invariants that constrain relationships among multiple interaction paths. These invariants express properties such as:

- **Precondition preservation:** Any path reaching a restricted state must include an authentication transition.
- **Path equivalence:** Distinct workflows implementing the same conceptual task must converge to equivalent states.
- **Forbidden transitions:** Certain state transitions must never occur under any path.
- **Validation blocking:** Invalid inputs must prevent progression across all execution paths.

Unlike traditional state invariants, these constraints relate multiple traces or paths through the model.

5. Relational Counterexample Generation. Given the inferred model and specified invariants, the framework systematically searches for path pairs or path segments that violate relational constraints. When violations are detected, the system produces: (i) minimal counterexample trace pairs, (ii) state differences highlighting divergence, (iii) reproducible event sequences.

6. Diagnostic Reporting. Finally, violations are reported as relational inconsistencies, enabling developers to reproduce workflow-level logical errors that would remain undetected under structural or single-path testing.

3.2 Approach Details

Describe the details of each component in this subsection

4 Experimental Evaluation

To assess the effectiveness of our behavioral consistency testing framework, we propose the following research questions to guide our evaluation:

- **RQ1 (Effectiveness):** Can the proposed framework automatically identify logical inconsistencies and behavioral flaws in generative Web UIs that are missed by traditional methods? This question focuses on the detection rate of issues like broken path equivalence and bypassed preconditions.
- **RQ2 (Comparison):** How does the proposed relational invariant approach compare to structural testing and visual regression baselines in terms of fault detection capabilities? We aim to quantify the improvement over existing techniques that focus on isolated states.

- **RQ3 (Scalability):** How does the framework perform as the complexity of the Web UI increases? We will analyze the scalability of model inference and invariant checking when applied to applications with complex interaction logic.

4.1 Evaluation Setting

We will evaluate the proposed technique using a controlled experiment involving representative Web UIs and injected faults.

Subject Systems. We will select a set of open-source Web applications that exhibit complex interactive logic, such as authentication flows, conditional navigation, and multi-step validation. These applications will serve as surrogates for generative UIs to ensure reproducible experiments.

Fault Injection. To measure detection rates, we will inject specific logic faults into these applications that mimic common errors in generative interfaces. These faults will include improper authentication enforcement, inconsistent validation handling, and incorrect redirects.

Baselines. We will compare our framework against two primary baselines:

Structural Automation: A standard crawling tool that verifies DOM correctness and control presence but lacks relational reasoning.

Visual Regression: A testing tool that validates UI appearance but does not verify logical constraints across interaction paths.

Procedure. For each subject system, we will first use the Interaction Explorer to collect execution traces. The framework will then infer a state-transition model and check for violations of relational invariants. We will record the number of true positives (valid bugs detected) and false positives to calculate the detection rate.

4.2 Evaluation Results

Present the results of individual research questions in this subsection. Use tables/figures to present the results. You need to discuss your observations and speculate the reasons for observations.

5 Related Work

We classify related research into three main categories: Model-Based Testing, Invariant-Based Testing, and AI-Driven Testing.

Model-Based Testing (MBT)

Model-based testing abstracts UI behavior into state models to systematically generate tests across interaction paths. Utting et al. [4] provide a comprehensive taxonomy of these approaches, differentiating between online and offline test generation. This taxonomy currently guides the architecture of industrial MBT tools like GraphWalker and ComMA (used by Philips), which require formal model definitions. Unlike these traditional methods that rely on manual specifications, our approach automatically infers behavioral state models from interaction traces.

Invariant-Based Testing

Mesbah et al. [3] introduced invariant-based testing to automatically verify properties of interface states against generic constraints. This method underpins the widely used Crawljax tool, which maps and tests dynamic AJAX applications, and influences modern “self-healing” test agents that detect DOM faults without brittle scripts. While effective for isolated state validation, this approach typically does not enforce consistency across multiple execution paths. Our work extends this by defining relational invariants that check for logic preservation between differing workflows.

Dynamic and AI-Driven Testing

Dynamic GUI testing explores execution paths at runtime to uncover behavioral issues. Galeotti et al. [2] extended this by combining search-based generation with dynamic symbolic execution. This technique is foundational for white-box testing tools like EvoMaster, which are essential for optimizing inputs in complex cyber-physical systems. More recently, Wang et al. [5] utilized Large Vision Language Models (LVLMs) to enhance testing coverage by “looking” at the UI. This represents the cutting edge of agentic frameworks like WebProber and AutoQALLMs; however, these tools primarily focus on structural and visual correctness rather than the deep relational logic our framework targets.

References

- [1] Juan Pablo Galeotti, Gordon Fraser, and Andrea Arcuri. 2014. Extending a Search-Based Test Generator with Adaptive Dynamic Symbolic Execution. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 421–424.
- [2] Juan Pablo Galeotti, Gordon Fraser, and Andrea Arcuri. 2014. Extending a Search-Based Test Generator with Adaptive Dynamic Symbolic Execution. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 421–424.
- [3] Ali Mesbah, Engin Bozdog, and Arie Van Deursen. 2011. Invariant-based automatic testing of modern web applications. *IEEE Transactions on Software Engineering* 38, 1 (2011), 35–55.
- [4] Mark Utting, Alexander Pretschner, and Benoit Legeard. 2007. A taxonomy of model-based testing. *Software Testing, Verification and Reliability* 17, 5 (2007), 297–312.
- [5] Siyi Wang et al. 2024. Leveraging Large Vision Language Models for Better Automatic Web GUI Testing. *arXiv* (2024). arXiv:2410.12157.